

Modern UDF Methods and

State-of-the-Art Exploration

Topic 16 — From NDF to the tutorial taxonomy

Matheus Levy de Lima Bessa

SIBGRAPI 2026 – Tutorial on Neural Unsigned Distance Fields

Where Block 1 left us

Learning / supervision

- ▶ NDF is trained with **ground-truth UDF values**;
- ▶ sparse and noisy observations remain difficult;
- ▶ distance regression does not directly supervise gradient quality.

Explicit reconstruction

- ▶ a UDF has no sign transition for standard Marching Cubes;
- ▶ NDF first projects a dense point cloud;
- ▶ a separate meshing stage then recovers connectivity.

Near-surface behavior

- ▶ locally, the exact UDF behaves like $u(t) = |t|$;
- ▶ the gradient reverses across the surface;
- ▶ projection therefore depends on a delicate learned surrogate.

Observations beyond points

- ▶ how can posed images supervise an unsigned field?
- ▶ how can rendering carry geometric information?
- ▶ how can newer priors improve the learned surface?

Block 2 starts from these limitations, not from a chronological list of papers.

NDF is the baseline, not the endpoint

The original NDF establishes the core neural-UDF pipeline:

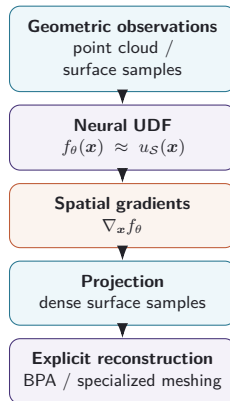
geometry $\rightarrow$ UDF regression $\rightarrow \nabla_x f_\theta \rightarrow$ projection $\rightarrow$ mesh

It demonstrated that a learned unsigned field can represent open and non-watertight surfaces without requiring a globally consistent inside/outside sign.

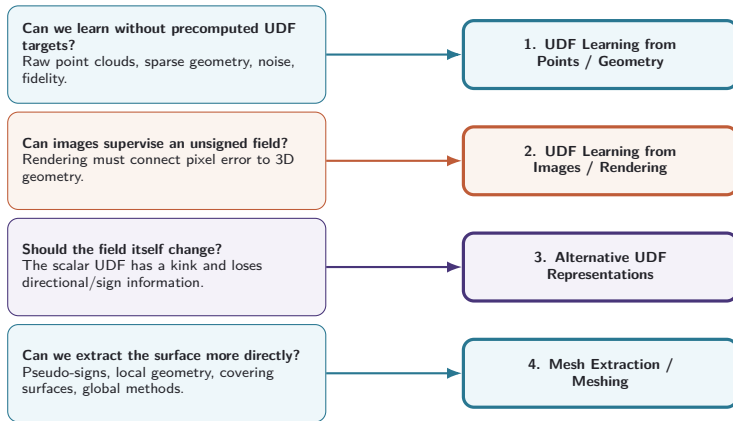
But each stage leaves a question

What if we change the **supervision**, the **observation model**, the **field representation**, or the **extraction procedure**?

Chibane, Mir & Pons-Moll, Neural Unsigned Distance Fields for Implicit Function Learning, NeurIPS 2020.



Four technical questions become four problem fields



These branches overlap in time: the taxonomy is organized by **technical problem**, not by year.

Companion survey taxonomy: Points/Geometry, Images/Rendering, Alternative UDF Representations, and Mesh Extraction/Meshing.

The tutorial taxonomy in one view

1. UDF Learning from Points / Geometry

- ▶ supervised UDF learning;
- ▶ raw point-cloud supervision;
- ▶ high-fidelity field learning.

2. UDF Learning from Images / Rendering

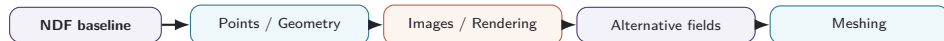
- ▶ canonical UDF volume rendering;
- ▶ Gaussian-assisted UDF learning.

3. Alternative UDF Representations

- ▶ hybrid signs / auxiliary labels;
- ▶ differentiable UDF surrogates;
- ▶ directional / factorized fields.

4. Mesh Extraction / Meshing

- ▶ pseudo-sign / grid-based extraction;
- ▶ dual / tangent-plane methods;
- ▶ double-covering methods;
- ▶ global / non-manifold / discrete extraction.



Companion survey, Sec. III: taxonomy of neural UDF methods.

Hands-on Representative-method

Laboratory

From research repository to interpretable UDF artifacts

Matheus Levy de Lima Bessa

SIBGRAPI 2026 – Tutorial on Neural Unsigned Distance Fields

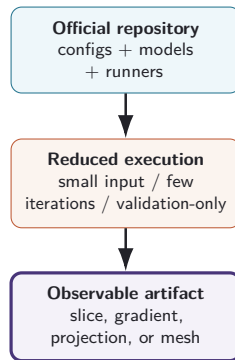
The laboratory goal is not to reproduce full papers live

The full research pipelines use different datasets, CUDA versions, compiled extensions, and long training schedules.

For the tutorial, we instead use a **reduced experimental contract**:

- ▶ inspect the official repository structure;
- ▶ identify where the field, losses, gradients, and extraction live;
- ▶ run a reduced or preconfigured experiment when practical;
- ▶ otherwise load a precomputed checkpoint or artifact;
- ▶ diagnose the result rather than waiting for full convergence.

Tutorial materials explicitly use reduced notebooks and precomputed artifacts so a local GPU is not required.



Four branches, four repository views

Taxonomy branch	Representative repo	Inspect first	Tutorial artifact	Slide
Points / Geometry	CAP-UDF	<code>run.py</code> , <code>confs/</code> , <code>models/</code> , Chamfer extension	raw cloud, projected queries, UDF slice, reconstructed mesh	22
Images / Rendering	NeuralUDF	<code>confs/</code> , <code>models/</code> , <code>loss/</code> , runner, custom MC	2D photos, UDF trained through the renderer, rendered vs. real photos	47
Alternative Representation	DUDF	<code>train.py</code> , <code>configs/</code> , <code>src/</code> , generated slice outputs	ordinary UDF vs. DUDF slice, gradient norm, curvature / normal cues	61
Meshing / Extraction	MeshUDF	pretrained UDF, <code>custom_mc/</code> , optimization script	same learned field, different extraction behavior / mesh artifacts	–

What we are learning

Repository inspection is used to connect the mathematical object from the slides to its **configuration**, **loss**, **intermediate output**, and **final geometry**. Each lab runs right after the slides on its method.

Selected official repositories: [junshengzhou/CAP-UDF](#), [xxlong0/NeuralUDF](#), [LIA-DiTella/DiffUDF](#), [cvlab-epfl/MeshUDF](#).

A common lab loop across very different codebases



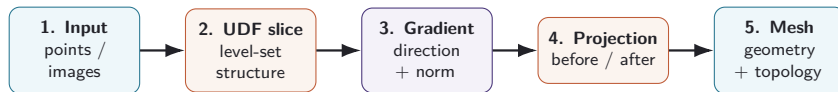
Reduced execution

- ▶ one shape / scene;
- ▶ tiny query batches;
- ▶ low grid resolution;
- ▶ checkpoint validation instead of full training.

Precomputed fallback

- ▶ checkpoint;
- ▶ 2D field slice;
- ▶ gradient visualization;
- ▶ projected cloud / mesh;
- ▶ known failure case.

The five visualizations we keep open during the laboratory



Why multiple views of the same run?

A low loss is not enough. A field can have plausible distances but poor gradients; projected points can collapse; a good implicit field can still produce a bad mesh.

What do we save?

For each branch, save a small artifact bundle: configuration, checkpoint identifier, one field slice, one gradient view, one geometry output, and one failure example.

The tutorial distribution includes checkpoints, field slices, gradient visualizations, projected point clouds, meshes, curves, and failure cases.

How to read the rest of Block 2

Representative methods in depth

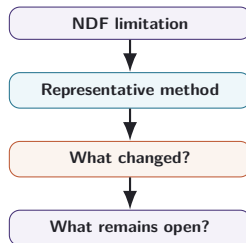
- ▶ **Points / Geometry**: CAP-UDF and GeoUDF;
- ▶ **Images / Rendering**: NeuralUDF and GaussianUDF;
- ▶ **Alternative representations**: DUDF and UODF;
- ▶ **Meshing**: MeshUDF and DCUDF.

Other papers stay in the reading map

They provide context and comparison without turning the tutorial into a paper-by-paper chronology.

For every branch, ask the same three questions

1. What limitation of the baseline is being addressed?
2. What part of the pipeline changes?
3. What assumption or failure mode remains?



Next: UDF Learning from Points / Geometry — from supervised distances to raw point-cloud learning.

Tutorial structure, Block 2: representative methods are organized by taxonomy branch and remaining works are retained as complementary reading.

UDF Learning from

Points / Geometry

Supervised UDFs, raw point clouds, CAP-UDF and GeoUDF

Matheus Levy de Lima Bessa

SIBGRAPI 2026 – Tutorial on Neural Unsigned Distance Fields

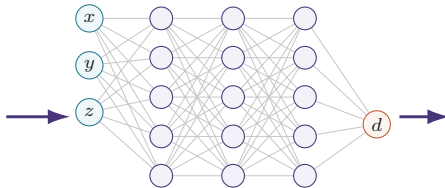
The setting: from a point cloud to a continuous field

Input: points



$\mathcal{P} = \{p_j\}_{j=1}^N \subset \mathbb{R}^3$
unordered, no connectivity, no normals

MLP f_θ



$f_\theta : \mathbb{R}^3 \rightarrow \mathbb{R}_{\geq 0}$
query $x \mapsto$ distance d

Output: continuous UDF



$f_\theta(x) \approx u_S(x), \forall x \in \mathbb{R}^3$
shells: points at equal distance

A discrete, finite sample goes in; a field that can be queried anywhere comes out.

Right: MLP trained on the Stanford bunny scan (bun000) — surface, iso-distance shells, and a front cut colored by distance.

The first branch asks: where does the geometric signal come from?

Same output, different supervision

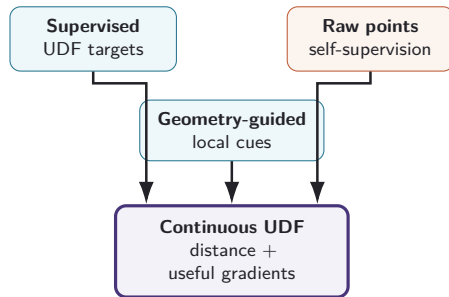
We still want a continuous unsigned field

$$f_{\theta} : \mathbb{R}^3 \rightarrow \mathbb{R}_{\geq 0}, \quad f_{\theta}(x) \approx u_S(x).$$

The key difference is **what information is available during learning**.

- ▶ **Supervised UDF learning:** train from precomputed unsigned distances.
- ▶ **Raw point-cloud supervision:** learn directly from the observed points.
- ▶ **High-fidelity field learning:** inject local geometric cues to improve distances and gradients.

Companion survey taxonomy: supervised UDF learning, raw point-cloud supervision, and high-fidelity field learning.



Supervised UDF learning: the NDF baseline

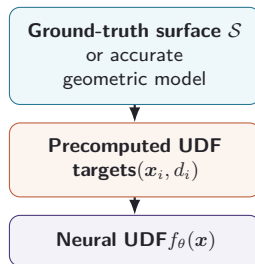
Given queries $\{\mathbf{x}_i\}$ and ground-truth unsigned distances $\{d_i\}$, NDF learns by direct regression:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N |f_{\theta}(\mathbf{x}_i) - d_i|$$

with $d_i = u_S(\mathbf{x}_i)$.

What supervision gives us

- ▶ a direct target at every sampled query;
- ▶ a simple and stable scalar regression objective;
- ▶ a clean way to learn the distance field around known geometry.



The dependency

Training requires access to the target UDF, not merely the raw observed point cloud.

Chibane, Mir & Pons-Moll, Neural Unsigned Distance Fields for Implicit Function Learning, NeurIPS 2020.

Raw point clouds remove the labels – not the geometry problem

What we observe

- ▶ a point set $P = \{p_j\}$ sampled from the surface;
- ▶ potentially sparse or noisy measurements;
- ▶ no precomputed UDF value for arbitrary queries.

What the method must invent

- ▶ a **self-supervised geometric objective**;
- ▶ a way to turn field predictions into a quantity comparable with P ;
- ▶ regularization that keeps the learned field geometrically meaningful.

What is missing

- ▶ the distance target $u_S(x)$;
- ▶ a direct constraint on near-surface gradients;
- ▶ explicit connectivity between points.

Hidden difficulty

The input points are only samples of the surface. The network must infer a **continuous field around them**, including useful behavior away from the samples.

Removing distance labels moves supervision from field values to geometric consistency.

Supervised vs. raw-point-cloud learning

	Supervised UDF learning	Raw point-cloud supervision
Observed data	geometry plus UDF targets	surface samples $P = \{p_j\}$
Training signal	direct distance regression	geometric consistency with observed points
Main advantage	simple, explicit scalar supervision	no precomputed ground-truth UDF
Main difficulty	obtaining accurate target distances	field and gradient quality must emerge indirectly
Representative method	NDF (2020)	CAP-UDF (2022)



CAP-UDF is the tutorial's representative example of this shift.

CAP-UDF: use the field to explain the point cloud

CAP-UDF replaces ground-truth UDF regression with a **consistency-aware self-supervised formulation**.

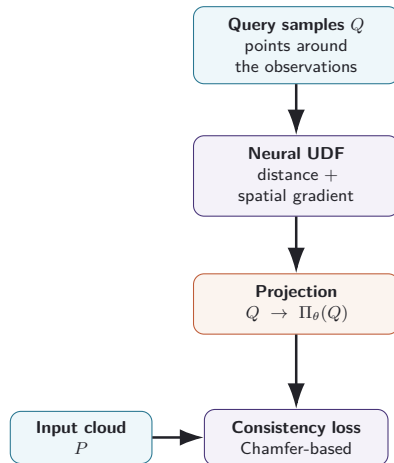
For a query x , the network predicts:

$$f_{\theta}(x) \quad \text{and} \quad \nabla_x f_{\theta}(x).$$

These quantities are used to move the query toward the predicted surface, reusing the projection idea developed in Block 1.

Key change

The projected samples are compared with the **input point cloud**, not with ground-truth unsigned distances.



Zhou et al., Learning Consistency-Aware Unsigned Distance Functions Progressively from Raw Point Clouds, NeurIPS 2022.

CAP-UDF turns projection into self-supervision

The projection operator from Block 1 has the conceptual form

$$\Pi_{\theta}(\mathbf{x}) = \mathbf{x} - f_{\theta}(\mathbf{x}) \frac{\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x})}{\|\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x})\|_2 + \varepsilon}.$$

Instead of supervising $f_{\theta}(\mathbf{x})$ with a target distance, CAP-UDF asks the projected queries to agree with the observed point set.

$$\mathcal{L}_{\text{self}} \sim \text{CD}(\Pi_{\theta}(Q), P)$$

Schematic tutorial form: the paper uses a consistency-aware, Chamfer-distance-based objective.

CAP-UDF, NeurIPS 2022; projection operator recalled from the NDF discussion in Block 1.

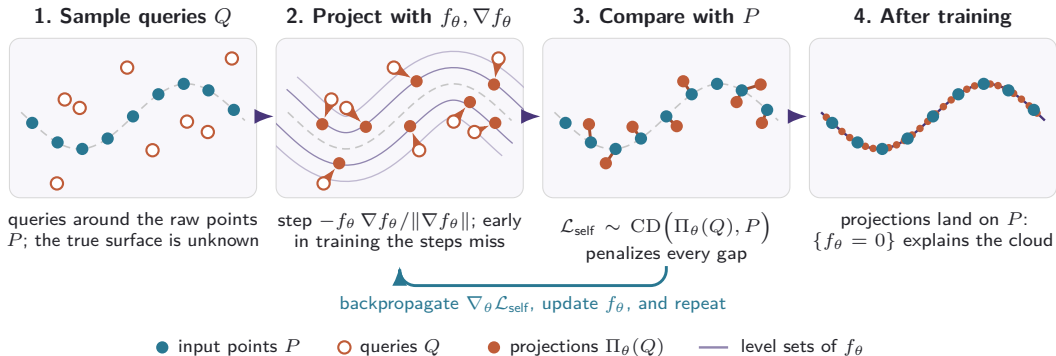
Why this is powerful

- ▶ no ground-truth UDF values are required;
- ▶ the learned field is constrained by the actual observation;
- ▶ projection and learning become part of the same feedback loop.

But notice the circular dependency

The loss becomes informative only if **distance and gradient predictions already move points sensibly**.

CAP-UDF at a glance: sample, project, compare



No distance labels: the only supervision is that projected queries coincide with the observed points.

CAP-UDF also adapts extraction – but gradients remain central

What CAP-UDF improves

- ▶ removes the need for ground-truth distance labels;
- ▶ learns from raw geometric observations;
- ▶ adapts Marching Cubes by using changes in gradient direction to identify crossings.

What remains difficult

- ▶ gradient quality is still not directly supervised;
- ▶ the point cloud is the geometric source of truth;
- ▶ sparse or noisy samples can destabilize the learned field.

Why that matters

The field is no longer trained as a purely supervised regression problem; **projection, consistency and extraction** are connected.

The next question

Can we inject stronger **local geometric information** so that distances and gradients are more faithful near the surface?

Next: Lab A, then inject local geometry to improve distance and gradient fidelity (GeoUDF).

Now to the notebook

► LAB A — CAP-UDF self-supervision

File: `sibgrapi_udf_block2_tutorial.ipynb`

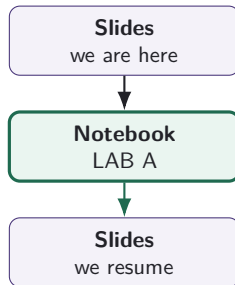
Notebook block: LAB A cells 4–28

What we run there

- a raw 2D point cloud P , with no distance labels;
- the projection $\Pi_{\theta}(Q)$, trained with $\text{CD}(\Pi_{\theta}(Q), P)$;
- the learned distance vs. the true one, and a densified cloud.

What to watch

Projections that overshoot or collapse reveal poor gradients, even when the distances look right.



↩ Back to slide 23

The hidden variable is gradient quality

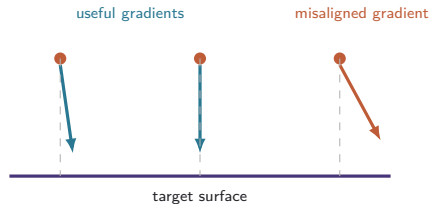
The hidden variable is gradient quality

Distance accuracy and gradient accuracy are related, but they are **not the same objective**.

Two fields can have similar scalar errors while producing different projection directions near the zero level set.

For projection-based methods

A small error in $\nabla_x f_\theta$ changes **where the sample moves**, even when $f_\theta(x)$ is numerically close to the correct distance.



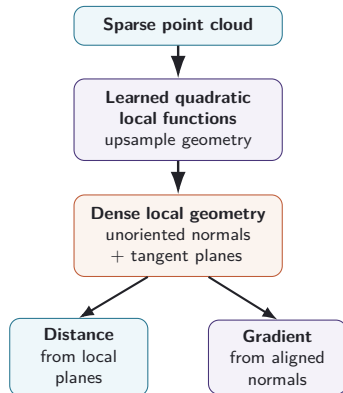
GeoUDF's response

Estimate local geometry explicitly and use it to improve **both distance and gradient information**.

GeoUDF: geometry-guided high-fidelity field learning

GeoUDF targets the regime of **sparse point clouds**, where nearest-sample geometry may be too crude. Its core strategy is to recover richer local structure before forming the field:

1. learn local quadratic functions to **upsample** the input;
2. estimate **unoriented normals**;
3. derive UDF information from neighboring **tangent planes**;
4. estimate gradients separately by combining aligned normals.



Design principle

Do not force all geometric information to pass through one scalar distance prediction.

Ren et al., GeoUDF: Surface Reconstruction from 3D Point Clouds via Geometry-Guided Distance Representation, ICCV 2023.

GeoUDF, step 1: every input point gets a neighbourhood

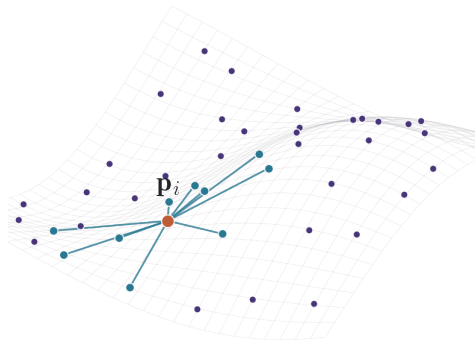
- ▶ Input: a sparse cloud $P = \{p_i\}_{i=1}^N$, **no normals**.
- ▶ For each p_i , take its $K = 10$ nearest neighbours.
- ▶ An EdgeConv encoder reads that neighbourhood and outputs **18 numbers**, reshaped into a matrix

$$A_i \in \mathbb{R}^{3 \times 6}.$$

What A_i is

The coefficients of one small **quadratic surface patch** around p_i : the network's local guess of the unknown surface.

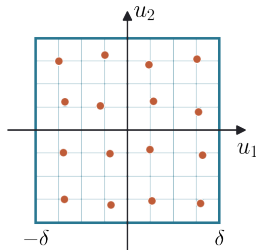
Ren et al., GeoUDF, ICCV 2023, Sec. 3.1. Figures: least-squares stand-in for the network on a synthetic surface.



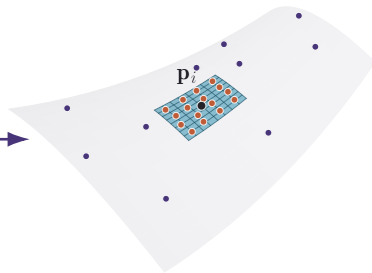
● p_i ● its K neighbours grey mesh: the unknown surface

Step 2: a local quadratic function maps a square onto the surface

$$\mathbf{f}_i(\mathbf{u}) = \mathbf{p}_i + \mathbf{A}_i E(\mathbf{u}), \quad E(\mathbf{u}) = \begin{bmatrix} 1 & u_1 & u_2 & u_1^2 & u_1 u_2 & u_2^2 \end{bmatrix}^\top, \quad \mathbf{u} \in D = [-\delta, \delta]^2$$



parameter domain D
 $M = 16$ samples $\mathbf{u}_m$



the patch $\mathbf{f}_i(D)$ around $\mathbf{p}_i$
and the lifted samples $\mathbf{f}_i(\mathbf{u}_m)$

A parametric patch, not a height field

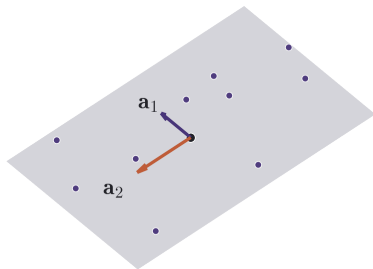
Every $\mathbf{u}$ in the flat square is sent to a 3D point. No local frame is fixed by hand: $\mathbf{A}_i$ decides where the patch sits and how it is oriented.

blue: the window D actually sampled;
grey: the same quadratic over the whole neighbourhood;
purple: the K neighbours it was fitted to.

Step 3: reading A_i column by column

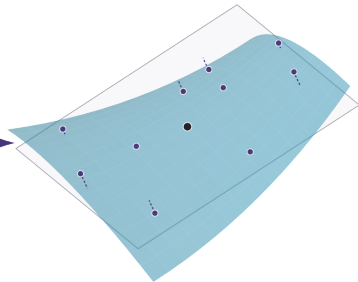
Write $\mathbf{a}_0, \dots, \mathbf{a}_5 \in \mathbb{R}^3$ for the columns of A_i :

$$f_i(\mathbf{u}) = \underbrace{\mathbf{p}_i + \mathbf{a}_0}_{\text{anchor}} + \underbrace{\mathbf{a}_1 u_1 + \mathbf{a}_2 u_2}_{\text{tangent plane}} + \underbrace{\mathbf{a}_3 u_1^2 + \mathbf{a}_4 u_1 u_2 + \mathbf{a}_5 u_2^2}_{\text{bending}}$$



First order: a plane through $\mathbf{p}_i$ spanned by $\mathbf{a}_1, \mathbf{a}_2$. It cannot reach the neighbours above and below it.

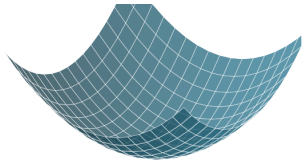
+ quadratic terms



Second order: the patch bends through the neighbours. Dashed: the gaps the plane leaves.

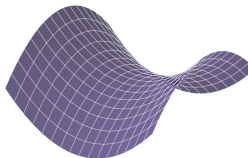
The constant column $\mathbf{a}_0$ lets the patch move slightly off a noisy $\mathbf{p}_i$. Here it is ≈ 0 .

Why quadratic? The shapes second-order terms can take



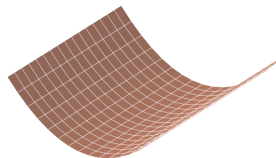
Bowl (elliptic)

both principal curvatures have the same
sign



Saddle (hyperbolic)

the curvatures have opposite signs



Ridge (parabolic)

flat along one direction

Along the normal $\mathbf{n}$, the bending terms read

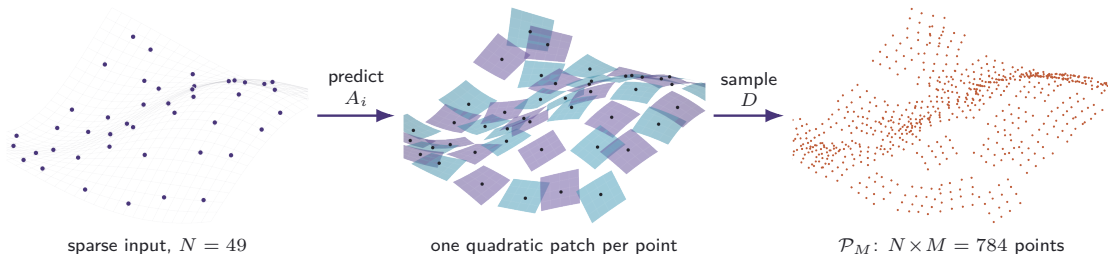
$$\langle \mathbf{n}, \mathbf{a}_3 \rangle u_1^2 + \langle \mathbf{n}, \mathbf{a}_4 \rangle u_1 u_2 + \langle \mathbf{n}, \mathbf{a}_5 \rangle u_2^2,$$

a local **second fundamental form**: three numbers that fix both principal curvatures. The tangential parts of $\mathbf{a}_3, \mathbf{a}_4, \mathbf{a}_5$ only change how the samples are spaced along the patch.

The lowest order that sees curvature

- ▶ a plane cannot bend;
- ▶ higher orders need more coefficients per point, fitted from only $K = 10$ neighbours;
- ▶ derivatives are closed-form, so normals are exact (step 5).

Step 4: upsampling means evaluating every patch at M samples



$$\mathcal{P}_M = \{ \mathbf{f}_i(\mathbf{u}_m) : i \leq N, m \leq M \}$$

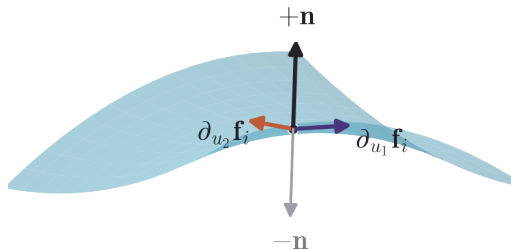
The $\mathbf{u}_m$ are spread over D by Poisson-disk sampling. Neighbouring patches overlap, so together they cover the gaps between input points.

Paper settings: $M = 16$, $\delta = 0.1$, $K = 10$.

Training signal

$\mathcal{L}_{\text{PU}} = \text{CD}(\mathcal{P}_M, \mathcal{P}_M^*)$: Chamfer distance to a dense ground-truth cloud. **Normals are never supervised.**

Step 5: normals come for free, but they are unoriented

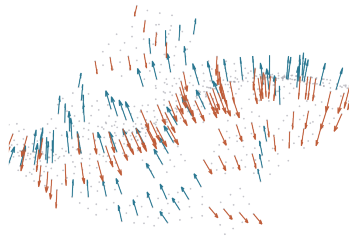


Differentiate the polynomial, not the network:

$$\partial_{u_1} \mathbf{f}_i = \mathbf{a}_1 + 2\mathbf{a}_3 u_1 + \mathbf{a}_4 u_2,$$

$$\partial_{u_2} \mathbf{f}_i = \mathbf{a}_2 + \mathbf{a}_4 u_1 + 2\mathbf{a}_5 u_2,$$

$$\mathbf{n} = \frac{\partial_{u_1} \mathbf{f}_i \times \partial_{u_2} \mathbf{f}_i}{\|\partial_{u_1} \mathbf{f}_i \times \partial_{u_2} \mathbf{f}_i\|}.$$



blue: $\mathbf{n}$ points up orange: $\mathbf{n}$ points down

Why the signs disagree

Swapping $u_1 \leftrightarrow u_2$ describes the same patch but flips $\mathbf{n}$. Each patch picks its own parameterization, so only $\pm \mathbf{n}$ is meaningful. The distance step uses $\pm \mathbf{n}$ only, so a UDF needs nothing more.

Why tangent planes are a better local cue than isolated points

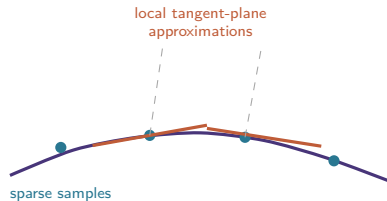
A sparse sample gives a location, but a local tangent model also gives **orientation information**.

Local geometric intuition

Near a smooth patch, a tangent plane approximates the surface better than a single point. GeoUDF uses neighboring tangent-plane information to derive local unsigned distances.

The estimated normals are unoriented, so the representation does not require a globally consistent inside/outside sign.

GeoUDF, ICCV 2023: denser local geometry, tangent-plane distances and separately estimated gradients.



Resulting effect

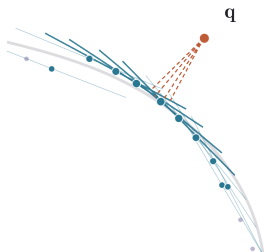
GeoUDF reduces the dependence of gradient estimation on the scalar distance prediction alone.

Step 6: the distance is an average of point-to-tangent-plane distances

For a query q , take its K nearest points $\{p_k\}$ in $\mathcal{P}_M$ and let $\tilde{p}_k = q - p_k$:

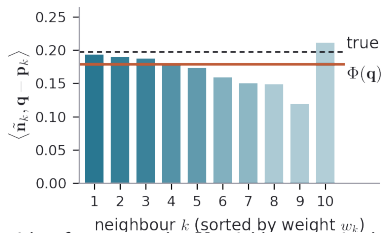
$$\tilde{n}_k = \text{sgn}(\langle n_k, \tilde{p}_k \rangle) n_k \quad (\text{so } \langle \tilde{n}_k, \tilde{p}_k \rangle \geq 0)$$

$$\Phi(q) = \sum_{k=1}^K w_1(q, p_k) \langle \tilde{n}_k, \tilde{p}_k \rangle$$



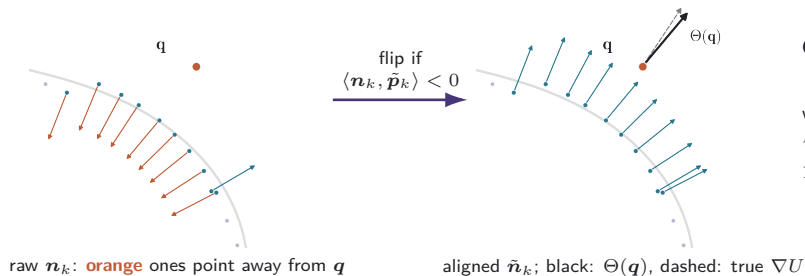
blue: the K nearest upsampled points and their tangent lines (larger dot = larger weight)

dashed: the distance from q to each tangent line



w_1 : an MLP with softmax over the K neighbours, trained with $|\Phi(q) - U(q)|$. In the figure, a distance-based stand-in.

Step 7: the gradient is a separate average of aligned normals



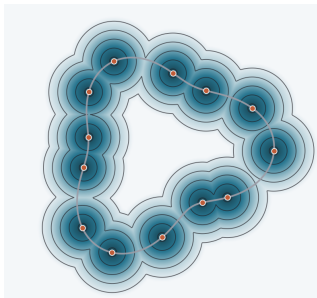
$$\Theta(q) = \frac{\sum_k w_{2,k} \tilde{\mathbf{n}}_k}{\left\| \sum_k w_{2,k} \tilde{\mathbf{n}}_k \right\|}$$

with its **own** learned weights
 $w_{2,k} = w_2(q, \mathbf{p}_k)$, trained with
 $1 - \langle \Theta(q), \nabla U(q) \rangle$.

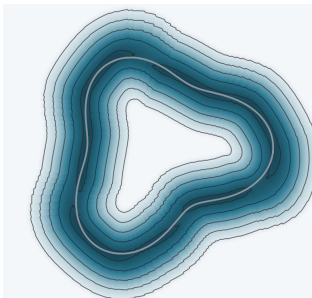
Why not just differentiate Φ ?

Θ never differentiates the scalar prediction, so small wiggles in Φ do not turn into wrong directions. This is the gradient that CAP-UDF-style projection and UDF mesh extraction rely on.

Why it works: the zero set becomes a surface, not a set of points



Distance to the sparse points
zero only at the samples, bumpy isolines



Point-to-tangent average on $\mathcal{P}_M$
isolines follow the curve

Same 13 input points in both panels;
darker means closer. Grey curve: the
true surface.

Mean error within 0.3 of the curve:

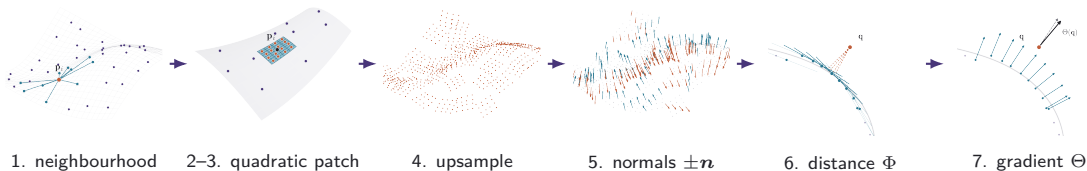
nearest sparse point	0.077
GeoUDF-style	0.017

(toy 2D example, stand-in weights)

Two ingredients

upsampling fills the gaps; tangent
planes flatten the bumps between
samples.

GeoUDF in one row of pictures



What the networks predict

Only coefficients and weights: A_i for each point, and w_1, w_2 for each query's neighbours.

What geometry provides

Where the points go, their normals, the distances and the gradient: all come from explicit local models.

The paper extracts the final mesh with an edge-based marching cubes (E-MC) that uses opposite directions of Θ to detect where a grid edge crosses the surface.

CAP-UDF and GeoUDF solve different parts of the branch

	CAP-UDF	GeoUDF
Primary question	Can a UDF be learned directly from raw point clouds?	How can sparse-point geometry yield more accurate distances and gradients?
Main mechanism	Projection + Chamfer-based geometric consistency	Local quadratic upsampling + normals + tangent planes
Role of gradients	Needed for projection and adapted extraction	Estimated with explicit local geometric cues
Main gain	Removes ground-truth UDF supervision	Improves near-surface geometric fidelity
Remaining issue	Sensitive to gradient quality and sparse/noisy observations	Final extraction remains grid based



Takeaway: supervision is only one part of field quality

What changes across the branch

- ▶ **NDF**: directly supervise unsigned distances;
- ▶ **CAP-UDF**: replace distance labels with point-cloud consistency;
- ▶ **GeoUDF**: strengthen local distance and gradient estimation with geometric structure.

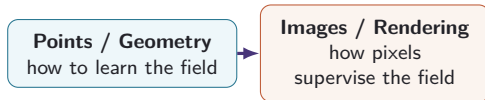
The progression is not simply “newer network = better network.” It changes **where geometric information enters the learning problem**.

Next: what changes when the observations are images rather than 3D points?

What still remains open

Even with stronger point/geometry learning, UDFs still face:

- ▶ intrinsic near-surface non-differentiability;
- ▶ difficult sparse/noisy observations;
- ▶ specialized downstream mesh extraction.



UDF Learning from

Images / Rendering

NeuralUDF, visibility-aware volume rendering, and GaussianUDF

Matheus Levy de Lima Bessa

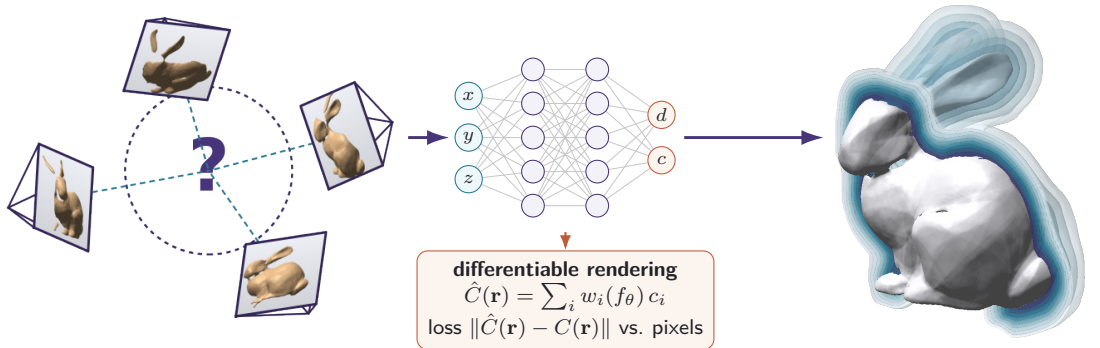
SIBGRAPI 2026 – Tutorial on Neural Unsigned Distance Fields

The setting: from posed images to a continuous field

Input: posed images

MLP f_θ

Output: continuous UDF



$\{(I_k, \pi_k)\}_{k=1}^K$: RGB views + cameras
same object, different viewpoints

$f_\theta(\mathbf{x}) \approx u_{\mathcal{S}}(\mathbf{x}), \forall \mathbf{x} \in \mathbb{R}^3$
recovered only through pixels

Only 2D observations go in; the same kind of 3D field comes out — a renderer links the two.

Illustration: left, views of the bunny from four posed cameras; right, the target UDF, as in the point-cloud setting.

The second branch changes the supervision source

From explicit 3D observations to pixels

In the previous branch, supervision came from distances or point-cloud geometry.

Here, the target surface must be recovered from **posed multi-view images**.

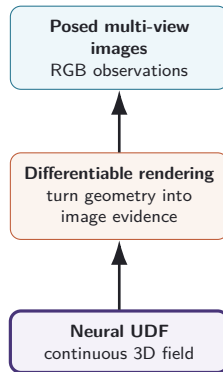
The network still represents a continuous UDF,

$$f_{\theta} : \mathbb{R}^3 \rightarrow \mathbb{R}_{\geq 0},$$

but image observations do not provide $u_{\mathcal{S}}(x)$ directly.

We therefore need a differentiable bridge from UDF values to image-supervisable quantities.

Companion survey, Sec. IV-B: image-based UDF learning optimizes the field indirectly through differentiable rendering.



The renderer is the missing supervision interface.

Why image supervision is fundamentally indirect

What the data gives us

- ▶ calibrated / posed images;
- ▶ pixel colors along camera rays;
- ▶ multiple views of the same scene.

What rendering must provide

- ▶ a way to transform UDF values into rendering weights;
- ▶ a differentiable path from rendered observations back to f_θ ;
- ▶ behavior that remains meaningful for open surfaces.

What the data does not give us

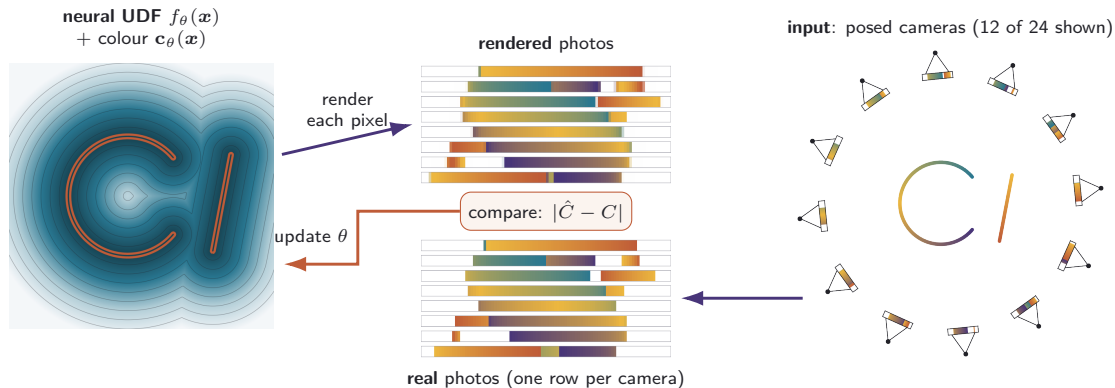
- ▶ UDF targets at arbitrary 3D queries;
- ▶ direct surface intersections for every ray sample;
- ▶ direct supervision of spatial gradients.

The central design question

How should a **nonnegative unsigned distance** become opacity, density, visibility, or another image-supervisable quantity?

The field is learned through the renderer, not from direct UDF labels.

NeuralUDF: learn a UDF by rendering it back into the photos

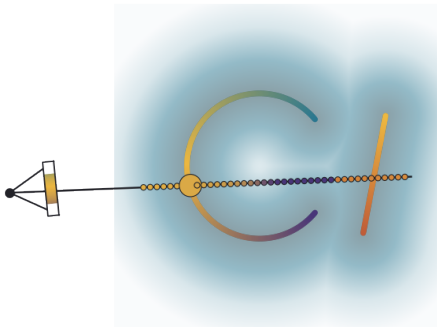


No 3D data goes in

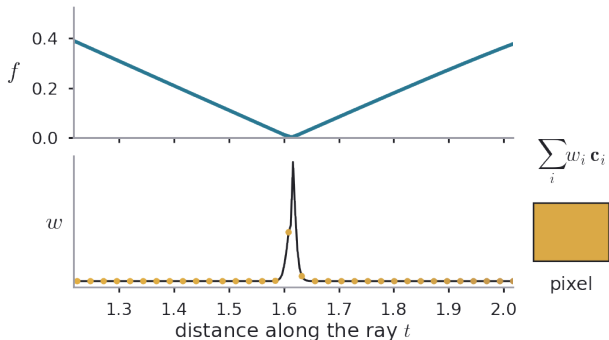
Only photos and camera poses. The UDF becomes whatever makes the rendered photos match the real ones.

Long et al., NeuralUDF, CVPR 2023. Figures: a 2D scene where each “photo” is a row of pixels; renders use NeuralUDF’s renderer, training snapshots are illustrative.

Rendering one pixel from the UDF



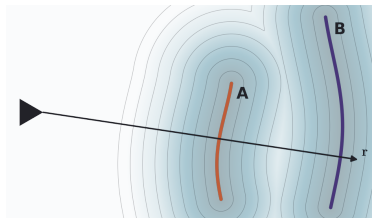
one pixel's ray; dots: samples, coloured by c , sized by weight



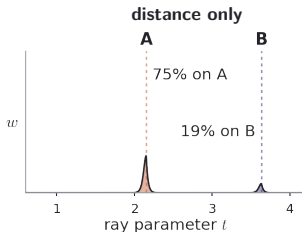
1. walk along the ray and query f and c at each sample;
2. turn distances into **weights** w_i , large where the ray first reaches $f = 0$;
3. the pixel is the weighted colour $\sum_i w_i c_i$.

Every step is differentiable, so the pixel error flows back into f and c .

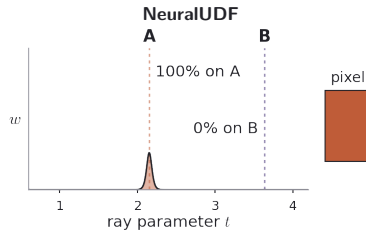
The one UDF-specific rule: a surface hides what is behind it



a ray that crosses two sheets



A lets light through, so the pixel mixes A and B



A is opaque, so the pixel is A's colour

Why an SDF does not have this problem

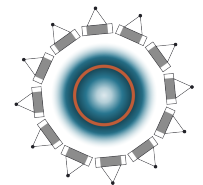
Its sign flips at the surface, which tells the renderer “you are now inside”.

What NeuralUDF adds

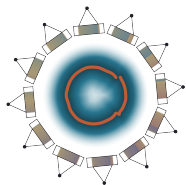
A **visibility indicator**: once the ray has passed a surface, the distance is read as “inside”, so everything behind it is hidden.

Training: optimizing the rendering loss shapes the UDF

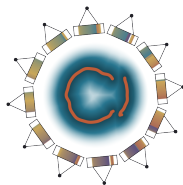
optimize f and c : render every camera, compare with its photo, update



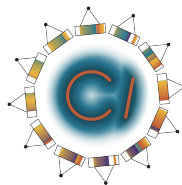
step 0
untrained UDF: a circle



step 250



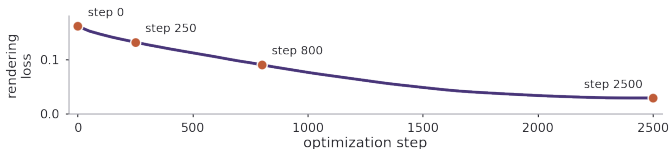
step 800



step 2500
learned UDF



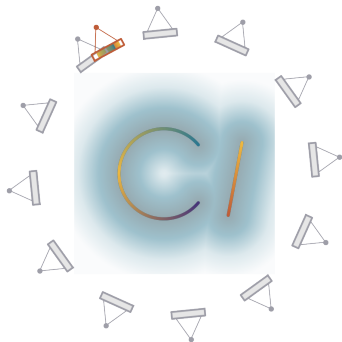
target
real scene and photos



Each camera (12 of 24 shown) displays what it **renders** at that step; **orange**: the zero set of f . Loss: $|\hat{C} - C| + \lambda(\|\nabla f\| - 1)^2$.

Illustrative sequence; every image is produced by NeuralUDF's renderer.

After training: render new views, extract the surface



orange: a camera that is not among the training views



top: its real photo; bottom: rendered from the learned UDF

- **New views:** render the UDF from any camera.
- **Geometry:** the zero set of f is the surface. In 3D, MeshUDF turns it into a mesh, open boundaries included.

What is designed by hand

The rule that turns f into weights. NeUDF and 2S-UDF revisit exactly this rule.

Now to the notebook

► LAB B — NeuralUDF: images to a UDF

File: `sibgrapi_udf_block2_tutorial.ipynb`

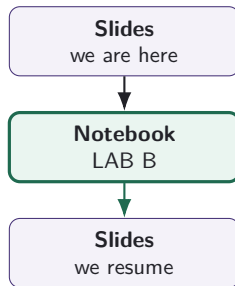
Notebook block: LAB B cells 29–48

What we run there

- 24 photos (rows of pixels) of a 2D scene, and their cameras;
- NeuralUDF's renderer, checked on the true UDF with and without Ψ ;
- the field and every rendered photo evolving during training, then a view never seen.

What to watch

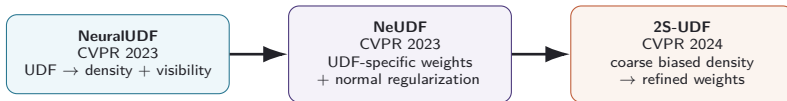
The photos are matched long before the shape is sharp, and what no camera sees stays ambiguous.



↩ **Back to slide 48**

Complementary volume-rendering developments

Complementary volume-rendering developments



NeUDF

Derives UDF-specific rendering-weight formulations and adds normal regularization to address orientation ambiguity on open surfaces.

2S-UDF

Uses a two-stage strategy: a slightly biased transparent density for coarse reconstruction, then direct refinement of rendering weights to make them unbiased and occlusion-aware.

These methods refine canonical UDF volume rendering; the tutorial keeps NeuralUDF as the main detailed example.

Why move from volume rendering to Gaussian-assisted UDF learning?

The survey motivates GaussianUDF with two limitations of the earlier image-based pipeline:

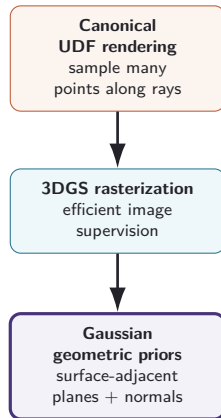
- ▶ **rendering inefficiency;**
- ▶ **weak geometric constraints.**

GaussianUDF replaces the canonical volume-rendering path with the differentiable rasterization of **3D Gaussian Splatting (3DGS)**.

Key conceptual shift

The image representation is no longer only a supervision mechanism: the Gaussians also become **local geometric priors** for UDF learning.

Companion survey: GaussianUDF uses 3DGS rasterization and addresses NeuralUDF's rendering inefficiency and weak geometric constraints.



GaussianUDF: Gaussians become geometric anchors for the UDF

GaussianUDF makes thin **2D Gaussian planes** lie on or near the surface and uses them as local geometric priors.

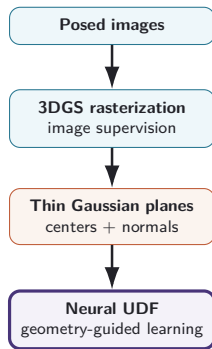
The survey highlights two different regimes:

- ▶ **near the surface:** use Gaussian normals instead of the UDF gradient;
- ▶ **far from the surface:** project query points toward the UDF zero level set and match them to Gaussian centers.

Why this matters

The UDF receives geometric guidance from the Gaussian representation instead of relying only on image-rendering weights.

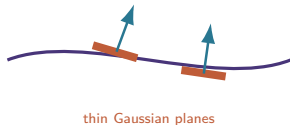
Li, Liu & Han, GaussianUDF: Inferring Unsigned Distance Functions through 3D Gaussian Splatting, CVPR 2025.



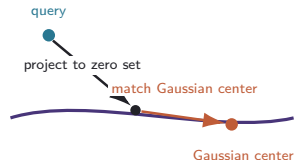
GaussianUDF uses different cues near and far from the surface

Near-surface region

Gaussian normals guide local geometry



Far-field region



Near the surface

Use **Gaussian normals** instead of the learned UDF gradient.

Far from the surface

Project queries toward the UDF zero level set and match projected points to **Gaussian centers**.

NeuralUDF vs. GaussianUDF: what changes?

	NeuralUDF	GaussianUDF
Image mechanism	canonical UDF volume rendering	differentiable 3DGS rasterization
UDF $\rightarrow$ image bridge	UDF-to-density mapping + visibility indicator	Gaussian-assisted image supervision
Geometric cue	Eikonal + iso-surface regularization	thin Gaussian planes, centers, and normals
Near-surface behavior	relies on the canonical UDF rendering formulation	uses Gaussian normals instead of UDF gradients
Far-field behavior	ray-based rendering weights	projects queries to the zero level set and matches Gaussian centers
Motivation highlighted by survey	enables multi-view UDF reconstruction of open surfaces	addresses rendering inefficiency and weak geometric constraints

The representation is still a UDF; what changes is how image evidence reaches the geometry.

Takeaway: image-based UDF learning is a rendering-design problem

NeuralUDF

- ▶ makes canonical UDFs trainable from multi-view images;
- ▶ maps UDF values to density;
- ▶ adds visibility reasoning around surface intersections;
- ▶ retains field regularization.

GaussianUDF

- ▶ moves image supervision to efficient 3DGS rasterization;
- ▶ turns Gaussians into local geometric priors;
- ▶ uses Gaussian normals near the surface;
- ▶ uses Gaussian centers for far-field consistency.

Images / Rendering
how observations
supervise a UDF



**Alternative UDF
Representations**
change the field itself

Next: instead of changing supervision or rendering, what if we change the UDF representation itself?

Alternative UDF

Representations

DUDF and UODF: beyond a single scalar unsigned distance

Matheus Levy de Lima Bessa

SIBGRAPI 2026 – Tutorial on Neural Unsigned Distance Fields

The third branch asks: is the scalar UDF itself the bottleneck?

Blocks 1 and 2 kept the same basic representation:

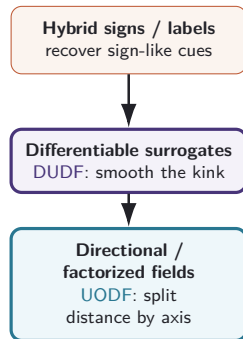
$$u_{\mathcal{S}}(\mathbf{x}) = \min_{\mathbf{p} \in \mathcal{S}} \|\mathbf{x} - \mathbf{p}\|_2.$$

But two intrinsic issues remain:

- ▶ **near-surface kink:** $u(s) = |s|$ is not differentiable at $s = 0$;
- ▶ **direction loss:** a scalar Euclidean distance says how far the surface is, but not *along which structured direction* it should be recovered.

Alternative representations change the field itself rather than only the loss, supervision source, or renderer.

Companion survey taxonomy: hybrid signs/labels, differentiable UDF surrogates, and directional/factorized fields.



Recall the UDF kink: the zero set is exactly where differentiability fails

Near a smooth surface point, use the signed normal coordinate s :

$$u(s) = |s|$$

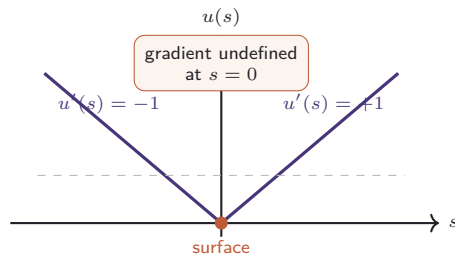
The one-sided derivatives are

$$u'(0^-) = -1, \quad u'(0^+) = +1.$$

The problem

A continuously differentiable neural field must approximate a cusp whose exact gradient does not exist at the surface.

DUDF, CVPR 2024; companion survey background on UDF non-differentiability.



The scalar values are well behaved; the derivative field is not.

DUDF: replace the cusp with a hyperbolically scaled UDF

DUDF does not learn the exact UDF directly. It learns the transformed field

$$t_{\mathcal{S}}(\boldsymbol{x}) = d_{\mathcal{S}}(\boldsymbol{x}) \tanh(\alpha d_{\mathcal{S}}(\boldsymbol{x}))$$

where $d_{\mathcal{S}}$ is the ordinary unsigned Euclidean distance and $\alpha > 0$ controls the smoothing region.

What is preserved

- ▶ the **same zero level set**: $t_{\mathcal{S}}(\boldsymbol{x}) = 0$ exactly on $\mathcal{S}$;
- ▶ nonnegative values;
- ▶ approximately linear distance behavior farther from the surface.

What changes

- ▶ the V-shaped cusp becomes **quadratic-like near zero**;
- ▶ the field becomes differentiable at the surface;
- ▶ $\nabla t_{\mathcal{S}} = 0$ on the zero level set.

Fainstein, Siless & Iarussi, DUDF: Differentiable Unsigned Distance Fields with Hyperbolic Scaling, CVPR 2024.

Hyperbolic scaling smooths only the region where the kink matters

For a 1D normal coordinate s :

$$d(s) = |s|, \quad t(s) = |s| \tanh(\alpha|s|).$$

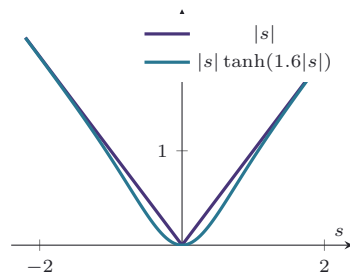
Near the surface,

$$\tanh(\alpha|s|) \approx \alpha|s| \quad \implies \quad t(s) \approx \alpha s^2.$$

Farther away,

$$\tanh(\alpha|s|) \rightarrow 1 \quad \implies \quad t(s) \approx |s|.$$

α controls how quickly the smooth surrogate returns to ordinary distance behavior.



Same zero set, smoother local differential behavior.

DUDF changes the Eikonal problem near the surface

The hyperbolic scaling induces a heterogeneous Eikonal equation:

$$\|\nabla f\| = \phi,$$

with surface conditions

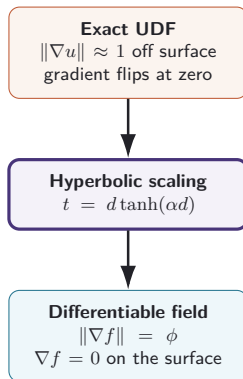
$$f|_{\mathcal{S}} = 0, \quad \nabla f|_{\mathcal{S}} = \mathbf{0}.$$

For the scaled distance,

$$\phi = \tanh(\alpha d) + \alpha d (1 - \tanh^2(\alpha d)).$$

This replaces the impossible requirement of a continuously differentiable UDF with unit gradient magnitude through the cusp.

DUDF, Sec. 3.2: hyperbolic scaling and the resulting Eikonal problem.



If the surface gradient becomes zero, where do normals come from?

At the zero level set, DUDF intentionally has

$$\nabla f(\mathbf{p}) = \mathbf{0}.$$

So the normal is *not* taken from the first derivative there.

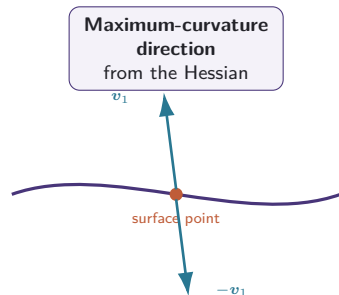
Instead, DUDF uses second-order information. Let

$$H_f(\mathbf{p})$$

be the Hessian, and let $\mathbf{v}_1$ be the eigenvector associated with the largest-magnitude eigenvalue. The paper shows that, at the surface,

$$\mathbf{v}_1 = \pm \mathbf{n}_S.$$

DUDF, CVPR 2024: maximum-curvature Hessian eigenvectors align with the unoriented surface normal.



The sign ambiguity is natural for unoriented/open surfaces.

Now to the repository

► LAB C — DUDF diagnostics

Repository: LIA-DiTella/DiffUDF

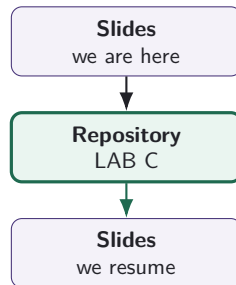
Inspect first: `train.py`, `configs/`, `src/`, generated slices

What we run there

- run a tiny configuration or load the supplied artifact;
- compare the distance and gradient-norm slices;
- compare the reconstructed meshes.

What to watch

Does smoothing the kink fix the gradient norm and reduce fragmentation?



↩ **Back to slide 62**

UODF asks a different question: can distance carry structured direction?

UODF asks a different question: can distance carry structured direction?

A conventional scalar UDF stores

$$u(\mathbf{p}) = \min_{\mathbf{q} \in \mathcal{S}} \|\mathbf{p} - \mathbf{q}\|_2,$$

where the closest point may lie in **any direction**.

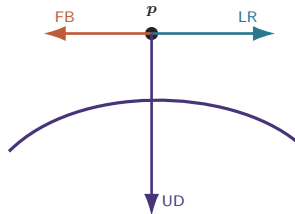
UODF factorizes this information along three orthogonal axes:

$$\text{UODFs}(\mathbf{p}) = \{u_{LR}(\mathbf{p}), u_{FB}(\mathbf{p}), u_{UD}(\mathbf{p})\}.$$

Each component asks for the nearest surface intersection **only along its own axis-aligned ray**.

Lu et al., Unsigned Orthogonal Distance Fields, CVPR 2024.

**Three factorized
1D searches**
instead of one
all-direction minimum



UODF definition: one unsigned distance field per orthogonal direction

For an orthogonal direction $X \in \{LR, FB, UD\}$, let $\text{ray}(\mathbf{p}, X)$ be the axis-aligned ray through $\mathbf{p}$, and let

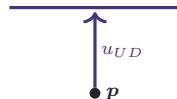
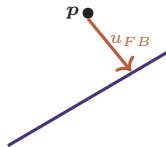
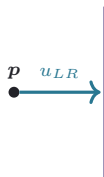
$$D(\mathbf{p}, X) = \mathcal{S} \cap \text{ray}(\mathbf{p}, X)$$

be its surface intersections.

When $D(\mathbf{p}, X) \neq \emptyset$:

$$u_X(\mathbf{p}) = \min_{\mathbf{q} \in D(\mathbf{p}, X)} \|\mathbf{p} - \mathbf{q}\|_2$$

and the component is undefined when that ray does not intersect the surface.



UODF, Sec. 3.1: three orthogonal directional fields computed from ray-surface intersections.

Why factorization helps reconstruction: each sample predicts a surface point directly

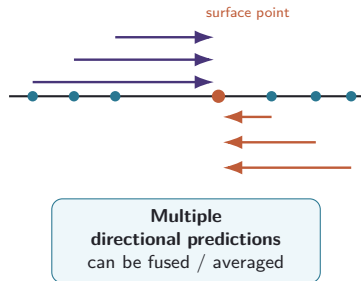
Along one orthogonal ray, UODF behaves as a collection of **1D unsigned distance fields**.

The paper highlights:

- ▶ along each valid ray, the absolute 1D derivative has magnitude 1;
- ▶ a sample point can directly estimate its nearest surface point along that ray;
- ▶ multiple samples can predict the same intersection and be averaged.

This avoids the extra linear interpolation step used to locate a grid-edge crossing from neighboring scalar SDF/UDF samples.

UODF, Secs. 3.2 and 3.5: 1D directional fields and direct surface-point estimation along rays.



UODF gains structure, but the factorization has its own trade-offs

Useful structure

- ▶ three orthogonal closest-point directions;
- ▶ direct surface-point prediction along rays;
- ▶ interpolation-free grid-edge point estimation;
- ▶ support for watertight and non-watertight shapes.

Extra machinery

- ▶ one UODF is fitted for each orthogonal direction;
- ▶ a mask is needed for rays with no surface intersection;
- ▶ the original formulation trains separate directional networks.

A distinctive property

Each directional field is continuous **along a ray**, but it may be discontinuous between neighboring parallel rays when the set of surface intersections changes.

Downstream extraction

The method reconstructs surface points first; mesh extraction is still a separate downstream stage.

UODF, CVPR 2024; companion survey notes the need to train three neural networks as a limitation.

DUDF and UODF attack different information bottlenecks

	DUDF	UODF
Primary problem	UDF non-differentiability at the zero level set	loss of structured directional information in one scalar distance
Representation change	hyperbolically scaled scalar UDF	three axis-aligned unsigned directional fields
Key local behavior	quadratic-like smoothing near the surface	1D distance behavior along each orthogonal ray
Surface geometry	normals recovered from Hessian maximum-curvature direction	surface points predicted directly along orthogonal rays
Main benefit	differentiable near-surface field and higher-order geometry	reduced interpolation error and structured surface recovery
Cost / limitation	second-order derivatives and extra differential constraints	multiple directional fields / networks and masks



Takeaway: alternative representations move the solution into the field definition

DUDF

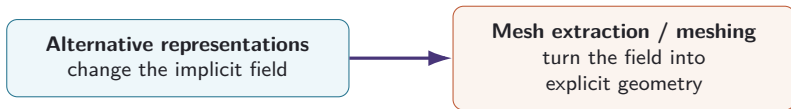
Instead of trying to make a neural network reproduce the exact cusp,

$$|s|,$$

learn a differentiable transformation with the **same zero set** and explicitly different derivative behavior.

UODF

Instead of compressing closest-surface information into one Euclidean scalar, keep **three directional distance channels** that directly support surface-point recovery.



Next: once the field is learned, how do we extract a robust mesh without a sign change?

Mesh Extraction /

Meshing

MeshUDF, DCUDF, and robust non-manifold extraction

Matheus Levy de Lima Bessa

SIBGRAPI 2026 – Tutorial on Neural Unsigned Distance Fields

Once the UDF is learned, meshing becomes the next bottleneck

A learned UDF gives a continuous scalar field

$$u(\boldsymbol{x}) \geq 0,$$

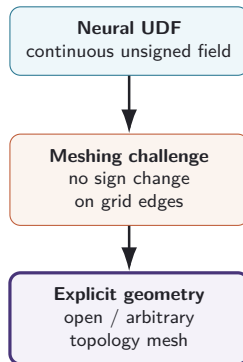
but not yet an explicit mesh.

For SDFs, sign changes let Marching Cubes locate the surface naturally. For UDFs, that sign information is missing.

Core question

How do we extract a robust mesh from an **unsigned** field while still handling open surfaces, sharp features, and even non-manifold structure?

This topic focuses on extraction, not field learning: how to convert UDFs into explicit meshes.



Why direct SDF-style extraction fails on a UDF grid

Sampling a UDF on a grid gives only nonnegative values. Even if a grid edge crosses the surface, both endpoints may have positive values.

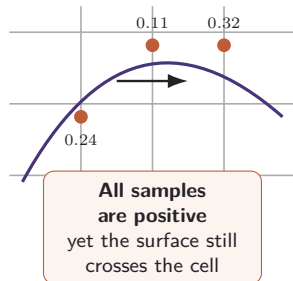
Consequence

A sign-based method cannot tell which side of the surface each sample belongs to, so the usual “inside/outside” topology test disappears.

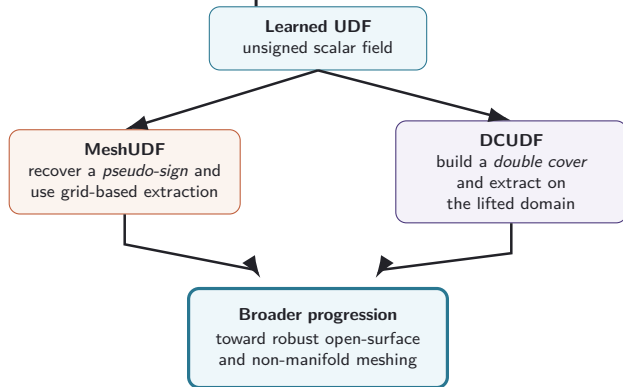
This creates three practical issues:

- ▶ **crossing ambiguity;**
- ▶ **orientation ambiguity;**
- ▶ **difficulty with open / non-manifold topology.**

This is the meshing analogue of the missing-sign problem seen earlier in rendering and optimization.



Two representative extraction responses in the UDF literature



MeshUDF intuition

Inject enough local orientation information to let a classical grid-based extractor behave as if a sign existed.

DCUDF intuition

Avoid forcing a single sign onto the original domain; instead, duplicate the domain so both local sides can be represented explicitly.

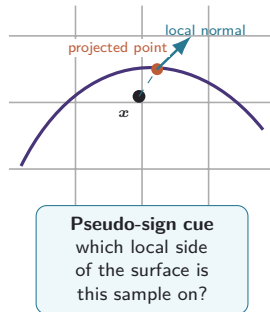
MeshUDF: pseudo-sign restores a sign-like extraction signal

MeshUDF keeps the efficiency of regular-grid extraction but augments the UDF with a **pseudo-sign**.

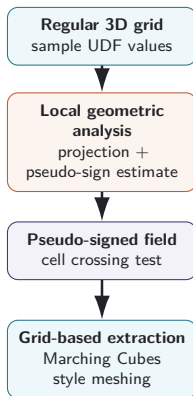
A local signed cue is estimated from geometric information around each sample, typically using a nearby projected surface point and its local normal / gradient information.

Idea

Turn an unsigned field into a **locally pseudo-signed field** so that a Marching-Cubes-style traversal can recover cells that cross the surface.
MeshUDF popularized the idea of sign emulation for UDF meshing.



MeshUDF pipeline: sample, assign pseudo-sign, extract



Strength

Simple, efficient, and easy to integrate with the familiar grid-sampling / Marching Cubes workflow.

Limitation

The sign is only *pseudo*; difficult configurations, non-manifold events, and thin structures can still remain ambiguous.

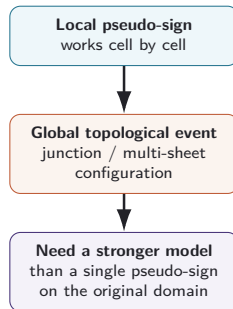
Pseudo-sign is a major step, but not the final answer

What MeshUDF resolves

- ▶ gives a sign-like cue for unsigned fields;
- ▶ enables regular-grid meshing;
- ▶ handles open surfaces better than a naive SDF assumption.

What still remains hard

- ▶ globally inconsistent local sign choices;
- ▶ cells containing multiple sheets or junctions;
- ▶ robust non-manifold extraction.



This motivates methods that change not only the field or the renderer, but the extraction domain itself.

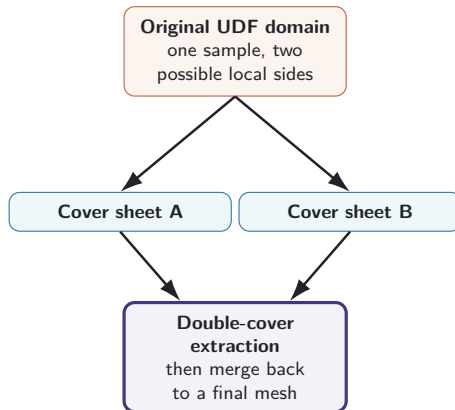
DCUDF: double-cover extraction avoids committing to one sign per point

DCUDF addresses meshing ambiguity by constructing a **double cover** of the domain.

Instead of assigning one sign-like state to each original sample, the method duplicates the domain into two coupled sheets. Intuitively, the two sheets represent the two local sides that an unsigned field cannot distinguish directly.

Key idea

Lift the extraction problem to a doubled domain where classical oriented extraction can be performed more robustly.



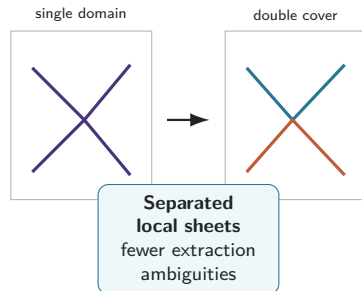
How the double cover helps with non-manifold and multi-sheet topology

On the original domain, a single sample may be locally compatible with more than one sheet configuration.

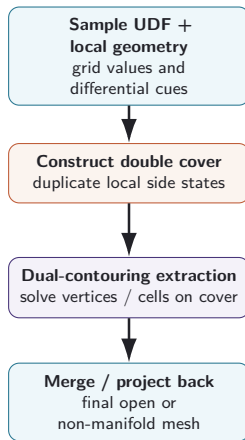
By working on a double cover:

- ▶ each local side can be represented separately;
- ▶ dual-contouring-style extraction can operate with oriented local decisions;
- ▶ the final mesh can contain **open surfaces and non-manifold events** more robustly.

DCUDF uses a double-cover construction to support robust dual-contouring-based extraction.



DCUDF pipeline: extract on the cover, then map back



Main message

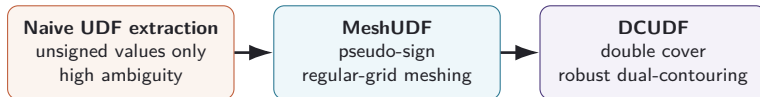
MeshUDF *emulates* sign; DCUDF *lifts* the extraction problem so sign-like topology can be handled in a richer domain.

MeshUDF vs. DCUDF: what changes in the meshing strategy?

	MeshUDF	DCUDF
Main response to missing sign	estimate a local pseudo-sign	duplicate the domain with a double cover
Extraction style	regular-grid / Marching-Cubes-style meshing	dual-contouring-style extraction on the cover
Topological philosophy	recover enough orientation to use classical extraction	explicitly represent both local sides before extraction
Main advantage	simple, efficient, compatible with grid pipelines	more robust for open, multi-sheet, and non-manifold structures
Main challenge	local pseudo-sign can remain ambiguous	more elaborate construction and optimization on the lifted domain

Both are milestones in the broader shift from “unsigned field” to “robust unsigned meshing.”

Broader progression: from sign emulation to robust non-manifold meshing



Trend

The literature gradually stops asking only “how do we find the zero set?” and starts asking “how do we recover the **right topology** from unsigned data?”

Outcome

This progression pushes UDF meshing beyond watertight assumptions toward open surfaces, junctions, and truly non-manifold geometric events.

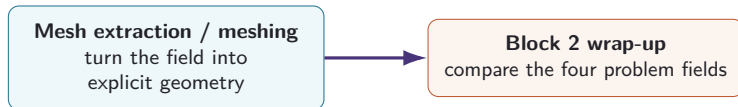
Takeaway: meshing is where unsigned geometry becomes explicit geometry

MeshUDF

- ▶ motivates pseudo-sign recovery;
- ▶ preserves the efficiency of grid-based extraction;
- ▶ shows how far sign emulation can go for open-surface meshing.

DCUDF

- ▶ motivates extraction on a double cover;
- ▶ separates local side states before meshing;
- ▶ connects UDF meshing to robust non-manifold extraction.



Block 2 closes by organizing the tutorial around supervision, rendering, representation, and meshing.

Taxonomy-level

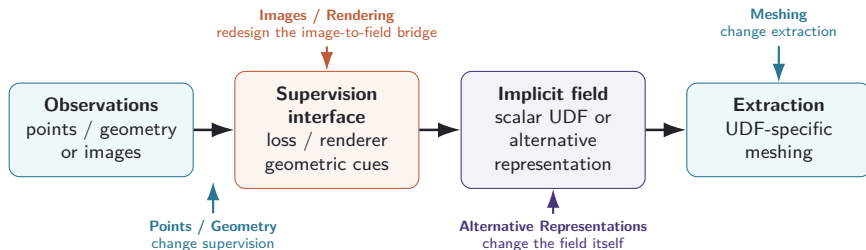
Comparison

Supervision, rendering, representation, and extraction

Matheus Levy de Lima Bessa

SIBGRAPI 2026 – Tutorial on Neural Unsigned Distance Fields

The taxonomy is organized by *where* a method intervenes



Taxonomy-level view

The four branches do not mainly differ by publication date. They differ by the **location of the intervention** in the reconstruction pipeline.

Companion survey: the taxonomy comprises Points/Geometry, Images/Rendering, Alternative UDF Representations, and Mesh Extraction/Meshing.

One-screen comparison of the four problem fields

Points / Geometry

Changes: supervision and geometric objective.

Strength: direct geometric constraints and raw-point learning.

Assumes: usable 3D observations.

Fails on: sparsity, noise, local geometry, gradient quality.

Images / Rendering

Changes: renderer and image-to-field supervision.

Strength: reconstructs from posed images without explicit 3D targets.

Assumes: calibrated views and a differentiable renderer.

Fails on: occlusion, view dependence, indirect geometry.

Alternative Representations

Changes: the mathematical field definition.

Strength: directly addresses kink / scalar ambiguity.

Assumes: the new field remains learnable and useful.

Fails on: added representation / optimization complexity.

Meshing / Extraction

Changes: field-to-mesh conversion.

Strength: explicit open / complex surface recovery.

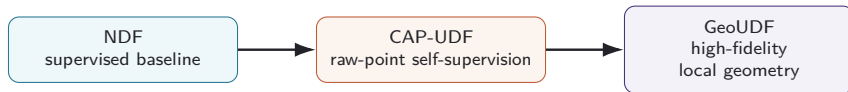
Assumes: reliable field samples and local geometric cues.

Fails on: local ambiguity, resolution limits, topology errors.

The same surface can fail for very different reasons depending on which stage is the bottleneck.

Points / Geometry: strongest when 3D evidence is available

What changes	Strengths / assumptions	Typical failure modes
▶ how the UDF is supervised; ▶ direct UDF targets vs. raw-point self-supervision; ▶ local geometric cues for distance and gradient quality.	▶ strong geometric grounding; ▶ can remove precomputed UDF targets; ▶ assumes useful point coverage and local neighborhoods.	▶ sparse or noisy clouds; ▶ unstable / inaccurate gradients; ▶ projection errors and fragmented geometry; ▶ difficult local curvature or detail.



Survey framing: supervised UDF learning, raw point-cloud supervision, and high-fidelity field learning.

Images / Rendering: supervision becomes indirect

What changes	Strengths / assumptions	Typical failure modes
▶ pixels replace explicit 3D distance targets; ▶ rendering turns UDF predictions into image-supervisable quantities; ▶ Gaussian-assisted methods change this interface again.	▶ learns geometry from multi-view images; ▶ supports open surfaces without a closed-surface assumption; ▶ assumes posed / calibrated images and a usable renderer.	▶ indirect geometric supervision; ▶ occlusion and visibility ambiguity; ▶ view dependence / illumination changes; ▶ weak local geometry or inefficient rendering.



Survey: image-only supervision is indirect and sensitive to occlusion, view dependence, and illumination changes.

Alternative representations: change the mathematical object

What changes	Strengths / assumptions	Typical failure modes
▶ the field is no longer the plain scalar UDF; ▶ smooth surrogates modify near-surface derivatives; ▶ directional / factorized fields store more structured information.	▶ directly attacks the UDF kink or scalar ambiguity; ▶ preserves open-surface motivation; ▶ assumes the new parameterization is learnable and compatible with downstream geometry.	▶ extra differential or architectural complexity; ▶ multiple fields / networks for factorized variants; ▶ reconstruction still depends on a downstream extraction stage.



Survey: alternative formulations improve differentiability or add information missing from a single scalar UDF.

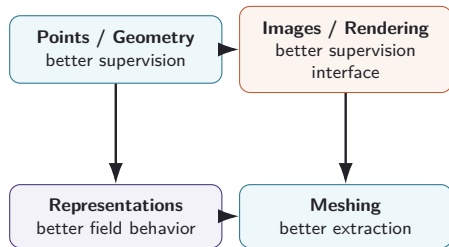
Meshing / Extraction: the field may be good while the mesh is still wrong

What changes	Strengths / assumptions	Typical failure modes
▶ the learned UDF can remain unchanged; ▶ the intervention is entirely downstream; ▶ extraction recovers surface structure without a standard sign transition.	▶ converts implicit UDFs into explicit meshes; ▶ can target open and non-manifold geometry; ▶ assumes reliable field samples, local crossings, gradients, or offset geometry.	▶ grid / sampling resolution limits; ▶ local crossing or pseudo-sign errors; ▶ dependence on gradient quality; ▶ topology mistakes in complex configurations.



Survey: UDF meshing progresses from local grid cues toward double-covering and global / non-manifold formulations.

No branch solves the full pipeline by itself



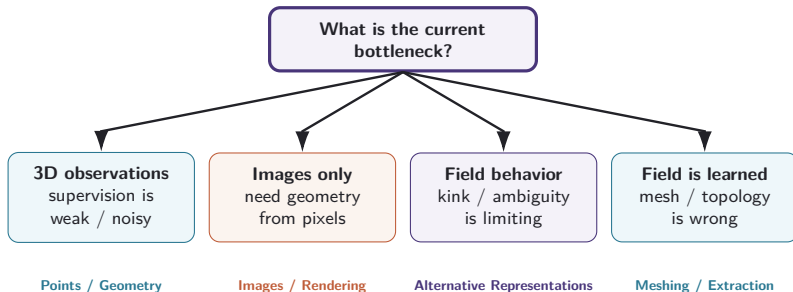
Cross-cutting dependencies

- ▶ gradients influence learning, projection, and extraction;
- ▶ image supervision still needs geometric inductive bias;
- ▶ alternative fields still need meshing;
- ▶ meshing cannot recover detail absent from the learned field.

Interpretation

Failures propagate downstream. A better extractor cannot fully repair a poor field, and a better field does not guarantee a robust mesh.

A practical decision guide for the four branches



Block 2 summary

Choose the branch by the failure mode you need to change: **supervision**, **rendering**, **representation**, or **extraction**.

Next: map the artifacts from Labs A–D back to the taxonomy.

Failure diagnosis: map the artifact back to the taxonomy

Observed artifact	First diagnostic	Likely bottleneck	Next thing to inspect
Projected queries oscillate / overshoot	gradient direction around the surface	Points / Geometry	field slice + local gradient vectors
Mesh is fragmented although distance looks good	near-zero gradient norm / consistency	Representation or learning	DUDF-style derivative diagnostics
Image reconstruction works but geometry is weak	rendering signal vs. geometric regularization	Images / Rendering	UDF slice, regularizers, visibility / sampling
Field looks correct but topology is wrong	extraction resolution / local crossing logic	Meshing / Extraction	same checkpoint with alternative extraction settings
Training is too slow to demonstrate	not a scientific failure	compute / engineering	switch to the precomputed artifact bundle

A useful lab result is a correct diagnosis, not necessarily a fully converged training run.

Laboratory takeaway: read research code through observable geometry

Do not begin with the full training command

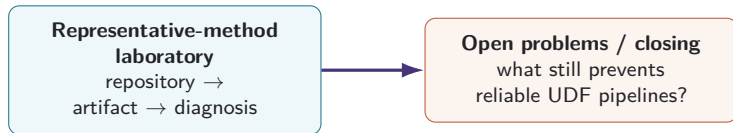
Begin with:

1. the representation output;
2. the loss / supervision signal;
3. the derivative or projection operation;
4. the extraction path.

End with a visual hypothesis

For every result, state:

- ▶ what artifact is visible;
- ▶ where in the pipeline it first appears;
- ▶ which taxonomy branch is responsible;
- ▶ what experiment would test that diagnosis.



The purpose of the lab is to make the taxonomy executable.

Open Problems and

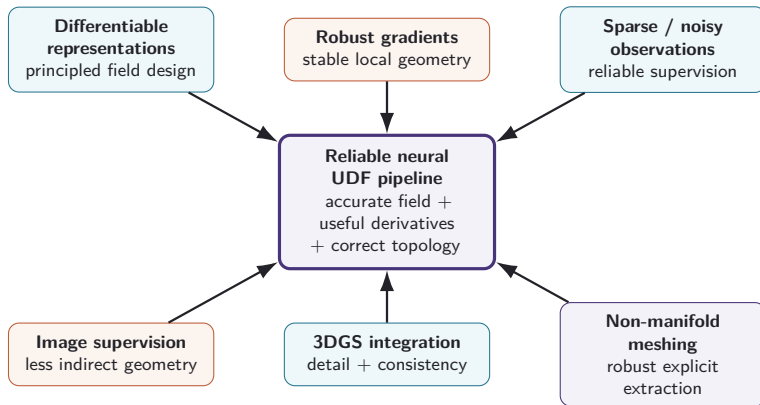
Closing Discussion

From differentiable fields to reliable non-manifold reconstruction

Matheus Levy de Lima Bessa

SIBGRAPI 2026 – Tutorial on Neural Unsigned Distance Fields

Six open fronts define the next stage of neural UDF research



The companion survey identifies these as the central remaining directions after reviewing UDF methods through 2026.

Open problem 1 — Principled differentiable UDF representations

The true UDF behaves locally like

$$u(s) = |s|,$$

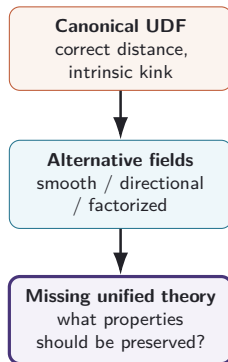
so exact unsigned distance is continuous but not differentiable at the surface.

Recent alternatives smooth, square, vectorize, or factorize the field. They improve different aspects of the problem, but they do not yet form one unified mathematical picture.

Open question

Can we define a representation with a clear theory connecting its zero set, derivatives, closest-point geometry, optimization behavior, and extraction?

Survey discussion: alternative representations improve differentiability through different reparameterizations, but a unified theory remains open.



Open problem 2 — Distance accuracy is not gradient reliability

A neural UDF can achieve low scalar distance error while still producing gradients that are:

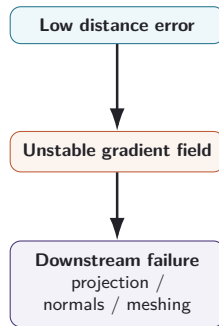
- ▶ vanishing near the zero level set;
- ▶ rapidly changing across nearby queries;
- ▶ inconsistent in direction;
- ▶ unreliable for projection or extraction.

This matters because gradients are used as geometric operators, not only as training diagnostics.

Open question

How should we train and evaluate UDFs so that gradient behavior is explicitly reliable where downstream algorithms need it?

The survey emphasizes that smooth neural approximations can have vanishing or inconsistent gradients even when UDF values are accurate.



Low value error does not guarantee a useful derivative field.

Open problem 3 — Sparse, noisy, and geometrically difficult observations

Sparse sampling

- ▶ large unsupported regions;
- ▶ weak local tangent estimates;
- ▶ fine structures can disappear;
- ▶ closest-point supervision becomes ambiguous.

Noise / outliers

- ▶ local normals become unreliable;
- ▶ projection targets drift;
- ▶ self-supervision can reinforce bad correspondences;
- ▶ gradient cues become inconsistent.

Complex local geometry

- ▶ high curvature;
- ▶ thin sheets;
- ▶ nearby surface branches;
- ▶ multiple plausible closest points.

Open question

How can a UDF distinguish missing evidence from actual empty space while remaining faithful to thin, sharp, or multi-sheet geometry?

Survey discussion: point-cloud UDF learning is comparatively mature, but sparse, noisy, and geometrically complex observations remain challenging.

Open problem 4 — Image supervision still reaches geometry indirectly

Image-based UDF methods must infer geometry through a renderer:

UDF $\rightarrow$ rendering weights / Gaussians $\rightarrow$ pixels.

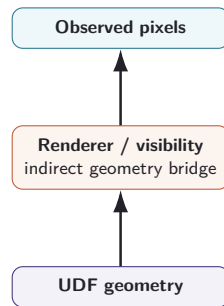
That supervision is affected by:

- ▶ occlusion;
- ▶ view dependence;
- ▶ illumination changes;
- ▶ the particular UDF-to-rendering formulation.

Open question

How do we make image supervision more geometry-aware without reintroducing strong closed-surface assumptions?

The survey notes that image-only supervision is indirect and sensitive to occlusion, view dependence, and illumination changes.



The training signal is photometric; the desired object is geometric.

Open problem 5 — 3DGS integration needs stronger geometric reasoning

3D Gaussian Splatting gives UDF pipelines an efficient differentiable rasterizer and useful local geometric anchors.

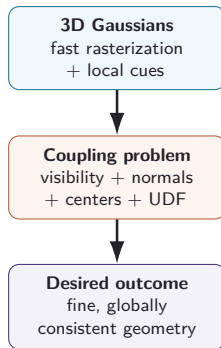
But the survey still identifies weaknesses in:

- ▶ local geometric reasoning;
- ▶ global optimization;
- ▶ preserving fine surface detail;
- ▶ maintaining consistency between Gaussian geometry and the UDF.

Open question

Can Gaussians become a principled geometric scaffold for UDF learning rather than only an efficient rendering representation?

Survey discussion: image-based UDF learning is increasingly coupled with 3DGS, motivating supervision that preserves detail and consistency.

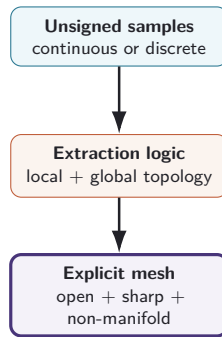


Open problem 6 — Robust non-manifold meshing remains central

Unsigned fields are attractive because they can represent open and arbitrary-topology geometry, but extraction must still recover that topology explicitly.

Future meshing must remain robust for:

- ▶ complex topology and junctions;
- ▶ non-manifold structures;
- ▶ noisy UDF predictions;
- ▶ discrete or sparsely sampled UDF data;
- ▶ sharp features and thin sheets.

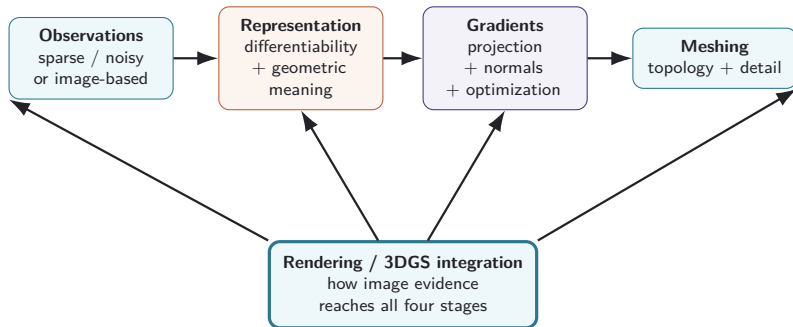


Open question

Can extraction become reliable enough that topology is a property of the learned geometry rather than an artifact of the meshing algorithm?

Survey discussion: robust and efficient extraction for complex topology, non-manifold structure, and noisy or discrete UDF data remains open.

The six problems are coupled, not independent



Research implication

Improving one stage in isolation can simply move the bottleneck. Reliable UDF systems need representations, supervision, derivatives, rendering, and extraction to be designed together.

What should future UDF evaluation measure?

Field quality

- ▶ distance accuracy;
- ▶ zero-level-set accuracy;
- ▶ behavior near thin / open structures;
- ▶ robustness under noise and sparsity.

Differential quality

- ▶ gradient direction;
- ▶ gradient magnitude;
- ▶ projection stability;
- ▶ local normal / curvature quality.

Explicit geometry

- ▶ completeness;
- ▶ geometric fidelity;
- ▶ boundary preservation;
- ▶ topology and non-manifold correctness.

Closing metric principle

A UDF should not be judged only by scalar regression error when the application ultimately depends on derivatives and explicit geometry.

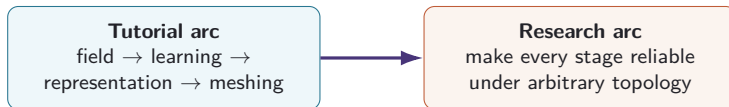
Closing discussion: six questions to take home

Representation and learning

1. What is the right differentiable analogue of an unsigned distance field?
2. How do we guarantee gradients that remain useful near the surface?
3. How much geometry can be recovered from sparse or noisy evidence?

Rendering and extraction

4. How should images supervise unsigned geometry directly?
5. What is the right division of labor between 3DGS and UDFs?
6. How do we extract open and non-manifold meshes robustly?



Final takeaway

Unsigned distance fields remove the need for a global inside/outside partition, but that freedom shifts difficulty into **differentiability, supervision, gradients, rendering, and meshing.**

The opportunity is not only to represent more surfaces.

It is to build reliable geometry pipelines for surfaces that signed fields were never designed to model.

Thank you. Questions and discussion.